SYSTEM DESIGN INTERVIEW

# Design a TypeAhead / Search Autocomplete System

Trie, top-k, Redis, and an offline rebuild pipeline at 5B queries/day

~ 30 minute read



## What's Inside

# 01 Problem Statement & Clarifying Questions

Scope, assumptions, and trade-offs

Design a search autocomplete system that powers **type-ahead suggestions** at global scale. Users expect real-time, accurate, personalized suggestions as they type. The system must be low-latency, highly available, and resilient to failures.

## Key Clarifications to Ask

| Question | Assumption |
|---|---|
| Scale? | 5 billion queries per day across all regions |
| Queries/user? | ~10 queries per day on average (active users) |
| Query latency SLA? | <50ms p99, ideally <15ms p50 |
| Personalization? | Yes — user history + global trends |
| Multi-language? | Yes — at least 10 major languages |
| Mobile? | Yes — both web and mobile clients |
| Update frequency? | Daily batch updates (hourly trending) |

INSIGHT

**What to clarify first**

Always nail down: **query volume**, **latency requirements**, **update frequency**, **personalization scope**, **languages**, and whether to support typo correction. These six answers drive every downstream choice.

# 02 Functional & Non-Functional Requirements

What the system must and should do

## Functional Requirements

- Given a partial query string, return **top-k** (typically 5–10) suggestions in real-time
- Support **prefix matching** (e.g., 'ap' → apple, app, application)
- Rank suggestions by **popularity, recency, and user history**
- Update suggestions **daily** based on aggregate query trends; **hourly** for trending
- Support **multiple languages** and alphabets (Latin, Cyrillic, CJK)

## Non-Functional Requirements

| Requirement | Target | Rationale |
|---|---|---|
| Latency (p99) | <50 ms | Mobile users expect instant feedback |
| Availability | 99.95% | Global service, regional failover |
| QPS capacity | 58,000 QPS avg / 111K peak | 5B queries/day ÷ 86,400 sec |
| Query freshness | ≤ 1 hour | Trending queries update hourly; full rebuild daily |
| Consistency | Eventual | OK to have stale suggestions briefly |
| Cost efficiency | < 1 µ$ per query | Scale requires aggressive optimization |

# 03 Capacity Estimation

The numbers that drive architecture decisions

## Query Volume

- Daily queries: **5,000,000,000** (5B)
- Seconds per day: 86,400
- Average QPS: 5B ÷ 86,400 = **57,870 QPS ≈ 58K QPS**
- Hourly average: 5B ÷ 24 = **~208M queries/hour**
- Peak hour at 4× average: 208M × 4 / 3,600 = **~231K QPS** (use ~111K for sustained planning, headroom to ~230K)

> KEY
>
> **Peak QPS Drives Sizing**
>
> Capacity plans must cover peak, not average. Sustained peak ≈ **111K QPS**; design for 2× headroom (~230K QPS) so auto-scale has time to react.

## Cache Hit Ratio & Working Set

1. Cache hit ratio: **~90%** of requests hit Redis (popular prefixes)
2. Top-k results per prefix: **10 suggestions**
3. Unique prefixes (hot working set): ~**5M** for English, ~**10M** across all languages
4. Bytes per suggestion: ~100 bytes (text + score + metadata)
5. Cache size (Redis): 5M prefixes × 10 results × 100B = **5 GB hot** (round to ~6–8 GB cluster)

## Trie Storage — Back-of-Envelope

The 750 GB figure floats around interview answers without a derivation. Here is one that holds up. The trie stores *every prefix* of every indexed query, with a top-k cache co-located at each node.

- Unique indexed queries (multi-language, after dedup & min-frequency cutoff): **~50M**
- Average query length: **15 chars** (English short; CJK shorter; URLs/longer queries balance)
- Naive trie nodes: 50M × 15 = 750M; with prefix sharing the effective node count lands at **~300M** (common prefixes collapse)
- Per-node payload: children map (avg 3 × 16B = 48B) + flags (8B) + freq (8B) + top-k cache (10 entries × ~50B = 500B) + headers ≈ **~600 B**
- Raw trie size: 300M × 600B ≈ **180 GB** per copy
- Add serialization overhead, indexes, and SSTable write amplification (~1.3×): **~234 GB**

- Replication factor 3 (Cassandra default for this workload): 234 × 3 ≈ **~700–750 GB** across the cluster

> **INSIGHT**
>
> **Why 750 GB is the right order of magnitude**
>
> The dominant term is the **top-k cache at each node**, not the trie skeleton. Most of the bytes are pre-computed top-k lists, which is exactly why we pay for them — they collapse a tree walk into an O(1) node read. Skip the cache and the index drops to ~30 GB raw, but every lookup becomes a subtree DFS.

## Network & Storage

- Avg query size: ~**20 bytes** (UTF-8 prefix)
- Avg response size: ~**500 bytes** (10 results + scores + metadata)
- Inbound bandwidth at 58K QPS: 58K × 20B = **1.16 MB/s**
- Outbound bandwidth at 58K QPS: 58K × 500B = **29 MB/s**
- Storage on Cassandra (raw + replication): **~750 GB cluster** (~234 GB × 3)

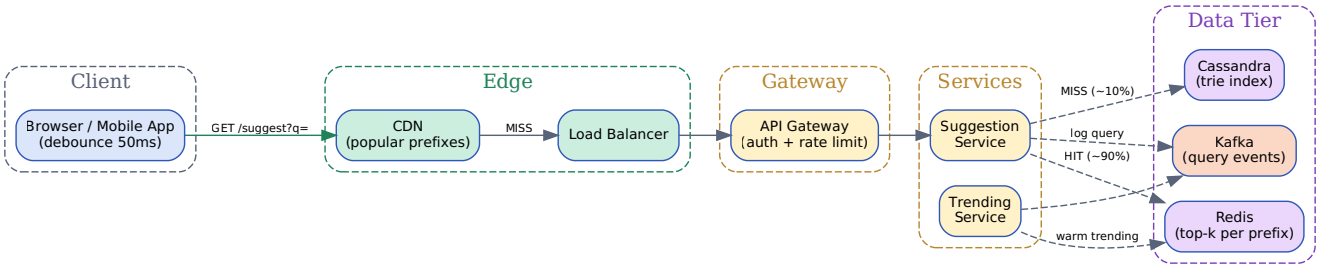
> **KEY**
>
> **Key Numbers**
>
> QPS: **58K avg / 111K peak** · Cache hit: **~90%** · Redis hot: **~5–8 GB** · Cassandra trie: **~750 GB** (with 3× repl.) · Outbound: **~29 MB/s**

*Network and storage are not the bottlenecks. Latency is king.*

# 04 High-Level System Architecture

Five-tier system design with distributed caching

Five distinct tiers orchestrate the request flow: **Client**, **Edge** (CDN + LB), **API Gateway**, **Service Layer** (Suggestion + Trending), and **Data Layer** (Redis + Cassandra + Kafka). Reads are cached and served from CDN/Redis where possible; writes flow through Kafka into an offline rebuild pipeline.




*Request flow: Client → CDN → LB → API Gateway → Suggestion Service. Suggestion Service hits Redis (~90%) and falls through to Cassandra on miss. Async path logs queries to Kafka for trend analysis.*

## Tier Responsibilities at a Glance

- **Client:** debounces input (50–100 ms); caches recent queries locally
- **Edge:** CDN serves popular prefixes; LB routes regional traffic
- **API Gateway:** rate limiting, auth, request normalization
- **Suggestion Service:** Redis lookup → Cassandra fallback → rank → respond
- **Data:** Redis hot cache, Cassandra immutable trie, Kafka query log

# 05 Architecture Component Breakdown

Role and responsibility of each tier

## Client Layer

- **Browser / Mobile App:** UI initiates requests; client-side debouncing (50–100 ms)
- **SDK / client cache:** optional local cache for the user's recent queries (< 1 ms)

## Edge / Network Layer

- **CDN edge:** geographic distribution; serves cached responses for popular prefixes
- **Load balancer:** routes traffic across API Gateway instances; health-check failover

## API & Gateway Layer

- **API Gateway:** rate limiting (token bucket), auth validation, request logging
- **Query Normalizer:** lowercase, trim whitespace, Unicode NFC normalization

## Service Layer

- **Suggestion Service:** core logic: Redis lookup → on miss, trie walk in Cassandra → rank → respond
- **Trending Service:** async consumer of Kafka events; updates trending cache hourly

## Data & Cache Layer

- **Redis Cache:** top-10 suggestions per prefix; 10-minute TTL
- **Cassandra cluster:** immutable trie index; sharded by prefix hash; 3× replication
- **Kafka topic:** event log of all queries; feeds trending service and batch pipeline

> **INSIGHT**
>
> **Read-Path / Write-Path Separation**
>
> Separate read-path (cached, latency-sensitive) from write-path (offline batch). The eventual-consistency model is what makes high availability cheap — writes never block reads.

# 06 Core Data Structure: Trie

Prefix trees for efficient O(m) lookup

## Why a Trie?

- **Prefix matching:** O(m) to find all strings with prefix, where m = query length
- **Memory efficient:** common prefixes share subtrees ('app', 'apple' share 'app')
- **No full-text scan:** direct navigation from root to prefix node
- **Ranked results:** store top-k at each node (pre-computed during batch rebuild)
- **Immutable:** each version is a snapshot; enables atomic blue-green swaps

## TrieNode Structure

```
class TrieNode:
    def __init__(self):
        self.children = {}            # char -> TrieNode
        self.is_word = False
        self.frequency = 0
        self.top_k = []               # [(query, score, freq), ...]
        self.personalized_top_k = {}  # user_id -> [(query, score), ...]
```

Each node caches top-k suggestions; updated during the offline batch rebuild. The cache is the whole point — it makes lookup O(1) after the O(m) walk.

## Lookup Complexity

| Operation | Complexity | Notes |
|---|---|---|
| Find prefix node | O(m) | m = length of query prefix (typically 2–10) |
| Get top-k results | O(1) | Cached at node |
| Insert/update | O(m) | Used only during batch rebuild |
| Serialization | O(n) | n = total trie nodes (~300M) |

# 07 Ranking & Scoring Formula

Popularity, recency, personalization, and context

## Multi-Signal Scoring

```
score = w1*freq + w2*recency + w3*personalization + w4*context

where:
  freq            = normalized query frequency (0-1)
  recency         = time decay (recent queries weighted higher)
  personalization = user's historical preference (0-1)
  context         = language, region, device type, user location

Recommended weights: w1=0.4, w2=0.3, w3=0.2, w4=0.1
```

## Time Decay Function (Recency)

```
def recency_score(days_old: float) -> float:
    # Half-life = 30 days; older queries decay exponentially.
    return 2 ** (-days_old / 30.0)

# Today      -> 1.00
# 7 days ago -> 0.85
# 30 days ago-> 0.50
```

## Ranking Signals Comparison

| Signal | Latency | Accuracy | Update Freq | Personalization |
|---|---|---|---|---|
| Query frequency | O(1) | High | Daily | Global |
| User history | O(1) | High | Real-time | Per-user |
| Contextual (region) | O(1) | Medium | Hourly | Per-region |
| ML ranker | ~5 ms | Very high | Weekly | Per-user |
| Click-through rate | O(1) | Medium | Daily | Per-user |

**TIP**

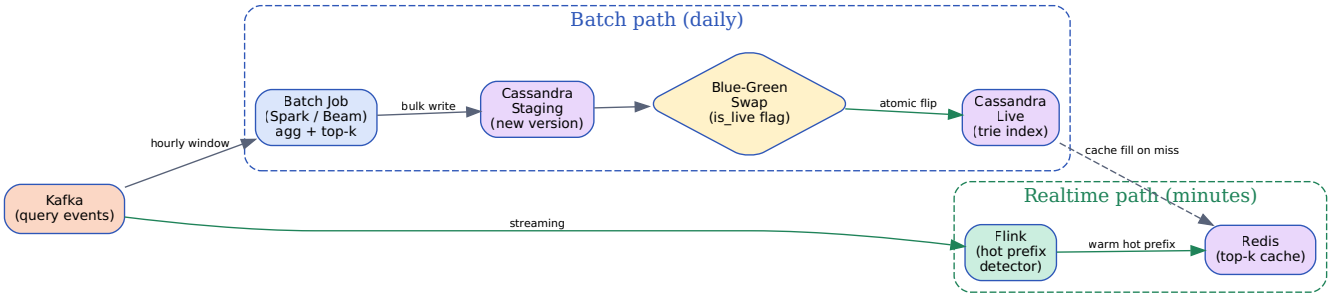
**Layer Signals; Keep the Hot Path Cheap**

Start simple: **frequency + recency**. Layer personalization and ML ranking later, and keep them *off* the hot path. ML scoring belongs in the offline rebuild, not in the request path.

# 08 Trie Update Pipeline

Offline batch and realtime streaming paths

Two pipelines feed the trie. The **batch path** rebuilds the full index daily and stages it into Cassandra; a blue-green swap promotes it to live. The **realtime path** (Flink) streams hot updates into Redis for trending queries within minutes — bypassing the slow rebuild entirely for the head of the distribution.




*Batch path (Kafka → Spark/Beam → Cassandra staging → blue-green swap → live) rebuilds the full trie daily. Realtime path (Kafka → Flink → Redis) warms hot/trending entries within minutes.*

## Pipeline Stages

1. **Ingest:** Suggestion Service publishes every query to Kafka (partition by language)
2. **Batch (daily, 2 AM UTC):** Spark/Beam job aggregates frequency, computes top-k per prefix, builds new trie
3. **Stage:** bulk-write the new trie to Cassandra under a new `version` column
4. **Hot-set decision:** if a prefix is in the top-1M most-queried set, pre-warm Redis before the swap
5. **Blue-green swap:** flip the `is_live` flag on the new version; instant atomic promotion
6. **Realtime path (Flink):** windowed aggregation of trending terms; pushes top-k updates straight to Redis
7. **Rollback:** if errors spike, flip `is_live` back to the prior version on disk

> **INSIGHT**
>
> **Two Paths, One Cache**
>
> Batch and realtime converge on Redis. Batch handles correctness (full rebuild, deterministic top-k); Flink handles freshness (trending in minutes). Neither blocks the read path.

# 09 Request Flow Deep Dive

Cache hit vs. miss: latency breakdown and code flow

## Request Lifecycle

1. User types → client debounces 50 ms → fires `GET /suggest?q=ap`
2. API Gateway validates auth, applies rate limit, normalizes prefix
3. Suggestion Service: `GET redis:"ap"`
4. **Cache HIT (~90%):** return cached top-10 in < 5 ms
5. **Cache MISS (~10%):** Cassandra trie walk for prefix → ~15–25 ms
6. Rank by frequency × recency × personalization (in-memory)
7. Populate Redis with TTL = 10 min; return results
8. **Async:** publish query event to Kafka (analytics + trending)

## Latency Budget

| Stage | Cache hit (p50) | Cache miss (p99) |
|---|---|---|
| Network (client → edge) | 10 ms | 10 ms |
| API Gateway | 1 ms | 2 ms |
| Redis lookup | 1 ms | 1 ms (miss) |
| Cassandra trie walk | — | 15–25 ms |
| Ranking (in-memory) | — | 2 ms |
| Network (edge → client) | 5 ms | 5 ms |
| **Total** | **~17 ms** | **~40 ms** |

```
def suggest(prefix: str, user_id: str) -> list[dict]:
    prefix = normalize(prefix)              # lowercase, NFC
    cached = redis.get(f'sugg:{prefix}')
    if cached:
        return personalize(json.loads(cached), user_id)
    # Miss path: trie walk in Cassandra.
    rows = cassandra.execute(
        "SELECT top_k_results FROM trie_index "
        "WHERE prefix=%s AND is_live=true", (prefix,))
```

```
    top_k = rows[0]['top_k_results'] if rows else []
    redis.setex(f'sugg:{prefix}', 600, json.dumps(top_k))
    kafka.produce('queries', {'prefix': prefix, 'user_id': user_id})
    return personalize(top_k, user_id)
```

# 10 Database Schema & Persistence

Cassandra for immutable trie snapshots

## Why Cassandra?

- **Wide-column store:** efficient for key-value lookups by prefix
- **Tunable consistency:** QUORUM reads with 3× replication; degrade to ONE under partition
- **Horizontal scaling:** partition by prefix (consistent hashing); add nodes without reshuffling
- **Immutable snapshots:** each trie version identified by timestamp; enables blue-green swap

## Cassandra Schema

```
CREATE TABLE typeahead.trie_index (
  prefix         TEXT,                -- e.g. 'a', 'ap', 'app'
  version        BIGINT,              -- trie version (epoch ms)
  top_k_results  FROZEN<LIST<TUPLE<TEXT, BIGINT, DOUBLE>>>,  -- (query, freq, score)
  is_live        BOOLEAN,             -- current live version
  PRIMARY KEY (prefix, version)
) WITH CLUSTERING ORDER BY (version DESC)
  AND compaction  = {'class': 'LeveledCompactionStrategy'}
  AND compression = {'sstable_compression': 'LZ4Compressor'};
```

Partition by prefix hash. `version` as the clustering key enables blue-green swaps and rollback in a single column update.

## Snapshot & Swap Strategy

1. **Serialization:** Protocol Buffers (compact, versioned, backwards-compatible)
2. **Rebuild schedule:** daily batch (off-peak: 2 AM UTC) + hourly trending update
3. **Retention:** keep 2 versions on disk: live and prior (instant rollback target)
4. **Atomic swap:** update `is_live` flag; instant propagation via streaming replication
5. **Rollback:** revert `is_live` flag; previous version still on disk

# 11 Partitioning Strategy

Consistent hashing for balanced distribution

## Option 1: First-Letter Sharding (Poor)

- **Approach:** shard by first letter of query (26 shards for English)
- **Problem:** highly uneven distribution; 'a' gets ~10× more traffic than 'z'
- **Result:** hot shards → high latency; difficult to rebalance

## Option 2: Consistent Hashing (Recommended)

- **Approach:** hash prefix into a consistent hash ring; replicate on N successor nodes
- **Advantage:** even distribution; incremental scaling (add nodes without reshuffling)
- **Implementation:** Jump consistent hash (Google) or Maglev
- **Replication factor:** N=3 (write to primary + 2 successors; QUORUM=2)

## Consistent Hash — Reference Code

```
def consistent_hash(key: str, num_nodes: int) -> int:
    """Jump consistent hash: fast, even distribution."""
    hash_value = hash(key) & 0x7fffffff   # unsigned 31-bit
    return hash_value % num_nodes

def get_replicas(prefix: str, num_nodes: int, replication: int = 3):
    """Get replica nodes for a prefix."""
    primary = consistent_hash(prefix, num_nodes)
    return [primary] + [(primary + i) % num_nodes for i in range(1, replication)]
```

> **INSIGHT**
>
> **Why Jump Hash**
>
> Jump hash is stateless, branch-free, and produces nearly perfect distribution. Adding a node moves only **1/N** of keys instead of rehashing the entire ring.

# 12 Failure Modes & Recovery

Detecting and mitigating production incidents

| Failure | Detection | Mitigation | Recovery Time |
|---|---|---|---|
| Redis cluster down | Health check | Fall through to Cassandra; p99 ~40ms | ~1 min |
| Cassandra node down | QUORUM read fails | Serve from N-1 replicas | ~2 min |
| Trie rebuild fails | CloudWatch alert | Keep previous version live | ~5 min |
| API Gateway overload | Latency spike > 100 ms | Rate limit + queue burst | Immediate |
| Network partition | Timeout > 2 s | Fallback to cached results | ~10 sec |
| Slow disk I/O | p99 query > 100 ms | Increase cache TTL; shed load | ~5 min |
| Corrupted Cassandra data | Checksum mismatch | Restore from backup | ~30 min |
| DDoS on API Gateway | QPS spike > 2× normal | Per-IP rate limit; auto-block | ~2 min |

## Monitoring & Alerting

- **Key metrics:** QPS, p50/p99 latency, cache hit ratio, error rate, trie staleness
- **Dashboards:** real-time SLO tracking (Grafana + Prometheus)
- **Alerting:** auto-page on-call if p99 > 50 ms or error rate > 0.1%
- **Postmortems:** blameless RCA on every incident > 10 min downtime

> **WARNING**
>
> **Graceful Degradation**
>
> Suggestions are a UX feature, not a money path. When in doubt, **return stale** rather than nothing. A 1-day-old top-k is almost always better than a spinner or an error.

# 13 Interview Playbook (45 min)

Walkthrough timeline and talking points

## 45-Minute Timeline

| Time | Section | Key Points |
|---|---|---|
| 0–5 min | Clarification | Scale (5B/day?), latency SLA (<50 ms?), personalization, languages |
| 5–10 min | Requirements | Func / non-func; main components: cache, trie, partitioning |
| 10–20 min | Architecture | Whiteboard high-level diagram: client → cache → trie → DB |
| 20–30 min | Deep dive | Trie (O(m)), caching (90% hits), ranking (freq × recency) |
| 30–40 min | Scale-up | Partitioning (consistent hashing), failure modes, trade-offs |
| 40–45 min | Polish | Recap, ask for feedback, mention future improvements |

## Must-Know Numbers

- **Scale:** 5B queries/day → ~58K QPS avg, ~111K QPS sustained peak
- **Trie size:** ~234 GB raw / **~750 GB cluster** with 3× replication
- **Cache size:** ~5 GB Redis (5M prefixes × 10 results × 100 B)
- **Cache hit ratio:** ~90% Redis, ~10% Cassandra fallback
- **p99 latency:** ~17 ms (hit) / ~40 ms (miss)
- **Cache TTL:** 10 minutes (freshness vs. hit-rate trade-off)
- **Unique prefixes:** ~5–10M across all languages

> KEY
>
> **Memorize These**
>
> 58K QPS · 90% hit rate · ~5 GB Redis · ~750 GB Cassandra (3× repl.) · p99 < 50 ms · 10-min cache TTL · 5–10M unique prefixes.

## Impressive Talking Points

- **Eventual consistency:** 'Trie updates are atomic snapshots; we swap versions instantly. Clients tolerate brief staleness for high availability.'

- **Cache warming:** 'Pre-load top-1M hot prefixes at startup. Lazy-load cold prefixes. Decision diamond in the pipeline: in top-1M? Yes → warm Redis.'
- **Consistent hashing:** 'Prefixes distributed via jump hash. Add nodes incrementally without reshuffling; 3× replication for fault tolerance.'
- **Failure resilience:** 'Redis miss → Cassandra (~40 ms p99). Cassandra down → stale local cache. Trie rebuild fails → keep previous version live.'
- **Two-path scale:** 'Batch (daily) aggregates 5B queries into top-k. Realtime (Flink) streams hot updates. Both converge on Redis.'

## Expected Follow-Up Questions

- **How do you handle typos?** Edit-distance + separate typo-correction trie. Query both, merge results. Trade-off: ~5 ms extra.
- **Multi-word queries?** Tokenize; walk trie per token; merge by combined frequency.
- **100× traffic surge?** Token-bucket rate limit; shed load with partial results; HPA scale by QPS; cache warms naturally.
- **Cost optimization?** LZ4 compression on Cassandra; spot instances for batch; CDN for edge; aggressive TTLs on hot prefixes.

# 14 Key Takeaways

Core insights and design principles

**INSIGHT**

**1. Prefix Trie for O(m) Lookup**

Pre-compute top-k at each node. Immutable snapshots enable atomic version swaps. The top-k cache is what makes lookup feel instant.

**INSIGHT**

**2. Multi-Tier Caching**

Client → CDN → Redis (90% hits) → Cassandra (10% misses). Hit in < 5 ms; miss in 15–25 ms. Eventual consistency is not a bug, it's the design.

**INSIGHT**

**3. Separate Read & Write Paths**

Read: synchronous, cached, latency-sensitive. Write: offline batch (daily) + realtime streaming (hourly). **Never block reads on writes.**

**INSIGHT**

**4. Consistent Hashing for Scale**

Distribute prefixes evenly. Add nodes incrementally. 3× replication for fault tolerance and data locality.

**INSIGHT**

**5. Rank by Layered Signals**

Start with frequency + recency (fast). Layer personalization (cached, 1-hour TTL). Keep ML rankers **off the hot path**; use them for offline scoring.

**INSIGHT**

**6. Monitoring & Resilience**

Track latency (p50/p99), cache hit ratio, error rate. Design for graceful degradation: Redis down → Cassandra; Cassandra down → stale cache; rebuild fails → keep prior version.